

Phase 19: Feedback Logger

Purpose

`19_feedback_logger.py` stores thumbs-up and thumbs-down feedback from users in a privacy-conscious format.

The phase uses the Phase 15 Streamlit transcript as a safe demo input and writes append-ready feedback artifacts for later quality review. It does not store hidden prompts, credentials, or raw private identifiers in comments.

Input Files

File	Purpose
<code>01_data/processed/15_streamlit_transcript.json</code>	Demo chat turns from the Streamlit UI evaluation.

Output Files

All generated files use the `19_` prefix so they line up with `19_feedback_logger.py`.

File	Purpose
<code>01_data/processed/19_feedback_log.json</code>	Full structured feedback records.
<code>01_data/processed/19_feedback_log.jsonl</code>	Append-friendly JSON Lines feedback log for future UI events.
<code>01_data/processed/19_feedback_report.json</code>	Summary counts for ratings, reason tags, answer modes, safety feedback, and thumbs-up rate.
<code>01_data/processed/19_feedback_audit.csv</code>	Spreadsheet-friendly feedback audit table.
<code>01_data/processed/19_failed_feedback_records.json</code>	Invalid feedback payloads, if any.

Plots Generated

Plot	Purpose
<code>01_data/processed/plots/19_feedback_ratings.png</code>	Shows thumbs-up versus thumbs-down counts.
<code>01_data/processed/plots/19_feedback_reason_tags.png</code>	Shows the most common feedback reason tags.

Code Section Guide

1. Configuration

`FeedbackLoggerConfig` stores the project root, transcript input path, output directory, and plot directory. Defaults match the project folder structure.

2. Dataclasses

`FeedbackRecord` defines the safe feedback schema: feedback ID, turn ID, request ID, rating, reason tags, sanitized comment, answer mode, safety flag, guardrail action, source count, and timestamps.

`FeedbackLoggerResult` returns every generated file path and headline counts.

3. Project Root Resolution

`resolve_project_root()` lets the Python script and notebook run from the workspace root, project root, `09_evaluation`, or `13_notebooks` folder.

4. Transcript Loading

`read_transcript()` loads `15_streamlit_transcript.json` and validates that it is a non-empty list.

5. Comment Sanitization

`sanitize_comment()` redacts email addresses, phone numbers, SSNs, and medical record numbers from optional feedback comments.

6. Feedback Record Creation

`create_feedback_record()` validates thumbs ratings and reason tags, sanitizes comments, copies safe answer metadata, and creates stable IDs like `19_FB_001`.

7. Sample Feedback Generation

`build_sample_feedback()` creates deterministic demo records for the Phase 15 transcript. Real UI code can call `create_feedback_record()` with live feedback payloads.

8. Writers and Plots

The phase writes JSON, JSONL, CSV, JSON report, failed records, and two plots: rating counts and reason-tag counts.

9. CLI Entry Point

Run from the project root:

```
python 09_evaluation/19_feedback_logger.py
```

or from the workspace root:

```
python hospital_patient_helpdesk_chatbot/09_evaluation/19_feedback_logger.py
```

Notebook and Python File Alignment

The notebook imports `09_evaluation/19_feedback_logger.py` and calls `log_feedback()` directly. That keeps the notebook and Python file compatible.

Difference Between .ipynb and .py

File	Role	Best Use
13_notebooks/19_feedback_logger.ipynb	Interactive walkthrough with feedback previews, validation, and plot display.	Learning, inspection, and manual review of feedback artifacts.
09_evaluation/19_feedback_logger.py	Reusable feedback logger and CLI workflow.	Automation, UI integration, and future monitoring pipelines.

The notebook is explanatory. The Python file is operational.

Safety Notes

- Do not store patient identifiers in feedback comments.
- Do not store hidden prompts, credentials, API keys, or system messages.
- Keep feedback focused on answer helpfulness, clarity, grounding, and safety.